Notițe și explicații pe care le-am considerat prea lungi ca să le fac comentarii

DEV LOG

Structura proiectului:

Flip.IT/:
/decks/ - conține fișierele CSV (câte un deck)
/main.cpp - meniul principal
/CSV.cpp/.h - manipulare CSV
/timp.cpp/.h - funcții legate de date (azi, scadent etc.)
/util.cpp/.h - funcții utilitare (ștergere ecran, input etc.)
/DOCS/README.docx - documentul principal
/DOCS/EXPLICATII_SUP.docx - explicații și notițe (fișierul actual)

Directivele (încep cu #) sunt executate în faza de preprocesare, înainte de compilarea codului C++. Ele permit includerea altor fișiere (#include), definirea de macro-uri (#define), evitarea includerii multiple a headerelor și compilare condițională (#if, #ifdef, #endif) - ce cod este inclus sau omis.

Un macro este definit cu #define și înlocuit în cod înainte de compilare, în faza de preprocesare. Oriunde apare numele macro-ului, preprocesorul îl înlocuiește cu valoarea/expresia asociată, este ca o „etichetă”.

Obs:
soluția pentru prevenirea includerii repetate a fișierelor .h scrisă în cod:
#ifndef HEADER (verifică dacă macro-ul HEADER NU a fost definit)
#define HEADER (dacă nu a fost, îl definește și îl copiază o singură dată în main.cpp)
...
#endif (finalul mecanismului)

> poate fi înlocuită doar cu directiva #pragma once

Multe funcții folosesc parametrii de forma: const (tip) &variabila – referințe constante. În loc să se copieze obiecte mari (ex: vector<Carte>), se transmite o referință constantă către variabila originală. Referințele constante îmi permit să folosesc aceași variabilă, dar fără să o pot modifica (keyword-ul const). => doar le citesc
La tablouri (int v[], char s[]...), numele se convertește automat într-un pointer către primul element. În schimb, containerele STL nu se comportă la fel, trebuie transmise prin referință dacă trebuie lucrate cu datele originale.

1) **/decks:**
Conține fișierele CSV ale deck-urilor. Separarea datelor este simplificată: câmpurile sunt despărțite prin „,”.
Fiecare linie reprezintă o carte, în ordinea: față, spate, văzut, termen, scor.
La prima rulare, dacă nu există niciun deck, se creează decks/default.csv.

2) **/main.cpp:**
Meniul principal. De aici sunt apelate funcțiile din modulele auxiliare:
util.cpp, CSV.cpp și timp.cpp, declarate în header-ele lor corespunzătoare (util.h, CSV.h, timp.h).
Programul selectează sau creează un deck inițial, îl încarcă, apoi rulează un meniu interactiv într-o buclă while(true) până la ieșirea utilizatorului. Fiecare opțiune de meniu apelează o funcție specifică pentru o acțiune (ex: învățare, salvare, ștergere, selectare deck etc.), pentru a nu fi prea aglomerat.

3) **/CSV.cpp/.h**
Funcțiile pentru gestionarea fișierelor .csv. Folosesc vector<string>, un vector de cuvinte (asemănător cu char m[][], dar nu trebuie să-mi fac griji de dimensiunile câmpurilor unei cărți).

a) vector<string> split(const string &s);

Am folosit un stringstream din biblioteca <sstream> pentru a separa și extrage fiecare subșir dintr-o linie citită din fișier. Un stringstream se comportă ca flux/stream de date de intrare și este folosit la fel cum ar fi folosit și std::cin.

L-am folosit cu getline cu sintaxa getline(stream, str, delim);> unde ss este fluxul de intrare (ca și cin.getline()), x string-ul în care salvez și sep este caracterul până unde este delimitat string-ul pe care vreau să-l citesc (implicit `\n`).

Pus într-o buclă while, împarte întregul stream de intrare și îl salvează într-un vector<string>.

Funcția împarte șirul s în câmpuri folosind delimitatorul sep (implicit).

b) void load(Deck &activ);

Încarcă în memorie cărțile deck-ului găsite la calea indicată de activ.cale.

Datele fiecărei cărți sunt citite ca string, iar conversia din string în int pentru scor este făcută cu stoi() care acceptă doar șiruri formate din cifre/numerice sau care încep cu cifre/au prefixul numeric, orice altă variantă dând o eroare(arucând o excepție) și ieșind din program.

try {...} catch (...) {...} verifică dacă scorul este un număr valid, prevenind oprirea programului în cazul unui scor invalid.

Metoda cu try-catch încearcă funcția stoi(), iar dacă eșuează, eroarea este prinsă de catch și tratată acolo, programul continuându-și execuția. În acest caz scorul este resetat la 0 și apare un mesaj corespunzător.

Catch(...) poate prinde orice excepție/eroare aruncată de try sau poate fi specificată catch(const tipul_de_eroare &e).

4) /timp.cpp/.h

Funcții legate de date calendaristice. Folosesc lib <ctime>, <sstream>, <cstdio>.

a) string azi();

Returnează data de astăzi într-un string cu formatul an-lună-zi.

time_t n = time(nullptr); > time(nullptr) returnează un timestamp current (numărul de secunde de la epoch – Unix time : 1970-01-01), salvat în variabila time_t n.

În C++ modern pe 64-bit, time_t este aproape întotdeauna un tip de dată de 64 biți (similar cu long long), dar depinde de platformă (pe sisteme vechi poate avea 32 biți).

tm t = *localtime(&n); > transformă timestamp-ul (numărul de secunde de la epoch, essential un șir lung de cifre) într-o structură human-readable. Câmpurile structurii tm t reprezintă diferite părți ale date actuale (ex: t.tm_yday = ziua dintr-un an, t.tm_hour = ora).

char timp[15];

strftime(timp, sizeof(timp), "%Y-%m-%d", &t); > string format time

return timp; > Funcția strftime salvează câmpurile specificate ale structurii într-un șir de caractere char timp[], conform șablonului specificat.

%Y = anul (4 cifre), %m = luna (01–12), %d = ziua din lună (01–31).

Șirul rezultat poate avea orice format, în funcția asta am ales ca anul, luna și ziua să fie despărțite de `-`.

b) string penalizare(const string &data, int zile);

Adaugă un număr de zile la data primită în format YYYY-MM-DD, și întoarce noua dată, tot în format YYYY-MM-DD (an-lună-zi).

int a,l,z;

char sep;

stringstream ss(data);

ss>>a>>sep>>l>>sep>>z; > la fel ca la funcția split(), dar aici este și mai evidentă similaritatea cu cin.

Operatorul >> funcționează aici exact ca la cin: extrage valorile în ordine în variabilele a, l, z, iar separatorul '-' este reținut în variabila sep.

Altă variant: sscanf(data.c_str(), "%d-%d-%d", &a,&l,&z);, dar trebuie string-ul să fie convertit în c string (const char*).
%d – înseamnă int

tm t={0}; > o structură calendaristică inițializată cu zero, e nevoie de ea mai târziu pt. a obține timestamp-ul
t.tm_year=a-1900; (t.tm_year stochează doar câți ani au trecut de la 1900, nu anul curent)
t.tm_mon=l-1; (lunile indexate de la 0 începând cu ianuarie)
t.tm_mday=z + zile; (adaug penalizarea)

time_t tt=mktime(&t) >mktime() transformă structura primită într-o data calendaristică validă
(ex: dacă data ar fi 2025-01-32, mktime() ar transforma-o în 2025-02-01 – doar că în formă de timestamp)

tm nt = *localtime(&tt)
char timp[15];
strftime(timp, "%Y-%m-%d", &nt);
return timp; > aceași explicație ca la azi()

> Separă YYYY-MM-DD în tm, adaugă zile în tm_mday, apoi mktime() normalizează data (gestionează depășirile de lună/an). Rezultatul este formatat cu strftime("%Y-%m-%d") și returnat ca string. Conversia char[] >string se face prin constructor, ca și la funcți azi().

5) /util.cpp/.h
Diverse funcții, utilități pentru meniul principal.

a) using fs = std::filesystem; > namespace alias, prescurtare

Folosesc prescurtarea doar când am nevoie de o funcție din acea categorie, fără a include tot namespace-ul (using namespace std::filesystem), care poate duce la conflicte cu alte funcții care au același nume. > am găsit un conflict: std::copy cu fs::copy

> Un namespace este folosit pentru a grupa funcții, clase/structuri, constante într-un spațiu de nume comun (o categorie) pentru a evita conflictul de nume identic dacă se folosesc mai multe biblioteci care au funcții denumite la fel. Un namespace poate fi creat și în cod pentru propria clasificare.

`::` > Scope Resolution Operator, accesează o funcție, clasă/structură sau constantă dintr-un anumit namespace/clasă.

b) cls();– șterge ecranul folosind o comandă de terminal în funcție de OS

În prima variantă, ștergeam buffer-ul stdout, dar mi s-a părut inutil.
buffer > pentru eficiență, cout (sau orice output stream) nu trimite caracter cu caracter, ci grupează mai multe ieșiri și le trimite împreună ca stream când are loc un flush.

Flush-ul se face automat la endl sau înainte de un cin și poate fi apelat manual cu cout.flush().
Dacă buffer-ul nu ar fi curățat înainte de cls/clear există posibilitatea ca ecranul terminalului să se curăță și de abia apoi să se afișeze ce a rămas în buffer.

> https://youtu.be/02rh1NjJLI4?si=mW-oU_ddmWS6Kdb3&t=192 > endl forțează și`\n` și flush, ceea ce poate încetini programele cu multe printări în buclă (nu e cazul aici).
std::cout<<endl = std::cout.put('\n') + std::cout.flush();
L-am eliminat, deoarece system("cls")/system("clear") oricum șterge tot ecranul.

Funcția finală folosește doar directive și macro-ul _WIN32 pentru a verifca pe ce platformă este compilat codul, folosind comanda potrivită. În funcție dacă _WIN32 este definit sau nu, alege ramura de cod.
(deși numele _WIN32 sugerează un OS de 32 biți, include și sistemele 64 de biți, este o convenție. Macro-ul _WIN64 există, dar este doar pt sistemele de 64 de biți).

c) ignora_linie();

Ignoră o line (tot fluxul până la `\n`). Citește într-un string toate caracterele până la '\n' și șterge '\n' din flux.

L-am folosit mai ales după citirea cu cin>>, deoarece lasă `\n` în buffer, care poate fi citit de getline() și returnează un string gol.

Cum există buffer pentru stdout, există și pentru stdin și sunt asemanătoare > nu citește caracter cu caracter, citește o linie întreagă până la new line '\n' într-un stream.

Alte opțiuni:
> char c;
while (cin.get(c) && c != '\n');
O altă opțiune viabilă, citește caracter cu caracter până la '\n', fără a salva tot streamul.
Este mai eficientă decât varianta cu string, deoarece nu alocă memorie.
cin.get() citea câte un singur char, de aceea trebuie pus în interiorul unei bucle.

>cin.ignore(numeric_limits<streamsize>::max(), '\n');
Are același efect, dar în loc să salveze toate caracterele până la '\n' le ignoră, inclusiv și '\n'
Este mai eficient, nu salvează varorile gunoi.

d) citiINT();

Obligă user-ul să introducă o valoare validă pentru un variabilă int.

Am scris mai multe variante. Cea actuală care folosește try{...}catch(...){...} și una care folosea cin.clear() pentru a reseta starea stream-ului.

i) Varianta actuală mi se pare mai simplă, folosesc aceeași logică ca și în funcția load(Deck &activ) din /CSV.cpp. Citește la nesfârșit un șir de caractere până când este introdus un șir care poate fi convertii într-o variabilă int sau până când primește un semnal EOF, caz în care returnează 0 și programul se sfârșește.

EOF – indică finalul stream-ului de date sau anumite combinații de taste (ctrl + Z, ctrl + C, ...)

ii) Codul vechi:

```
int citINTvechi()
{
    int x;
    while(true)
    {
        if (cin >> x)  // dacă citirea a reusit
        {
            cin_flush(); // se curăță buffer-ul - trebuie mușai, dacă nu
                         //-> eroare imd la următoare, se citește opt, rămâne '\n' și anulează actiunea
            return x;  // se returnează imediat valoarea validă
        }

        // altfel -> eroare
        cin.clear();        // se resetează starea fluxului
        cin_flush();        // se curăță buffer-ul
        cout << "Introdu un numar valid: ";
    }
}
```

Programul citește o valoare de la tastatură și, dacă intrarea nu este un număr valid,
cin intră într-o stare de eroare. În această stare, stream-ul nu mai poate citi nimic/ se blochează programul.

Pentru a nu se bloca programul, apelez funcția cin.clear() care resetează starea de eroare (readuce la goodbit) și flag-urile interne (failbit,badbit,eofbit).
Fără cin.clear(), fluxul rămâne blocat și nu mai acceptă inputuri.

cin.clear() - spune compilatorului că în cazul unei erori, programul continuă, eroarea fiind adresată de cod

FAILBIT - datele nu sunt de tipul cerut

BADBIT - problemă I/O, de exemplu vreau să citesc dintr-un fișier inexistent

eofbit - finalul stream-ului de date sau anumite combinații de taste (ctrl + Z, ctrl + C, ...)
(end of file)

iii) Mic experiment:

```
int tst()
{
    int x;
    cin.exceptions(ios::failbit | ios::badbit | ios::eofbit); //face ca cin>> să arunce exceptii
    while (true)
    {
        try
        {
            cin >> x;
            cin_flush(); //curăță newline-ul rămas
            return x;    //succes -> iese din functie
        }
        catch (const ios_base::failure& e)
        {
            cin.clear();   //resetează starea fluxului
            cin_flush();   //curăță bufferul
            cout << "Introdu un numar valid: ";
        }
    }
}
```

Deși failbit, badbit și eofbi sunt erori, dar ele nu sunt tratate ca unele. Sunt flaguri tratate de stream, în cod nu aruncă excepții care pot fi prinse în structuri precum try{...}catch(...){...}.

Dacă vreau să folosesc un astfel de mecanism, fără condiții suplimentare (if(cin>>x)) trebuie specificat obligatori că
cin aruncă excepții: cin.exceptions(ios::failbit | ios::badbit | ios::eofbit);

În rest, codul funcționează ca cel de mai sus.